Complexity Analysis - USEI12

Objectives:

"User Story Goal: As an infrastructure planner, I want to compute a minimum network over the rail network (treated as undirected) so that I can identify a minimal backbone that connects all stations with minimum total track length."

In algorithmic terms, it means it's supposed to develop a *Minimum Spanning Tree* of the whole Belgian Railway Network. A *Minimum Spanning Tree* is a spanning tree of an undirected graph that connects all its nodes with a subset of edges with minimum possible total weight - which means that the graph must be weighted - composing then a tree formed by the cheapest way to connect all its vertices. It doesn't contain cycles - it's not possible to go from one node and get back to it without repeating nodes and edges - and the number of edges is equal to the number of nodes minus 1.

The computation of this tree is possible through the implementation of the *Prim's Algorithm*.

Complexity Analysis:

The complexity of the Prim's Algorithm methods are based on the number of nodes/vertices *n* and number of edges *e*.

***primMST* method from *Algorithms* class**		
Code line	**Complexity**	**Description**

`for (int i = 0; i < n; i++) {`
`  dist[i] = Double.POSITIVE_INFINITY;`
`  path[i] = -1;`
`  visited[i] = false;`
`}`
	O(*n*)	For loop that defaults the arrays *dist*, *path* and *visited*.

`for (Node vAdj : adj) {`	O(*deg(n)*)	For loop that checks for

***primMST* method from *Algorithms* class**		

`int adjKey = g.key(vAdj);`
`double distance = g.edge(v, vAdj).getDistance();`
`if (!visited[adjKey] && dist[adjKey] > distance) {`
`dist[adjKey] = distance;`
`path[adjKey] = vOrig;`
`}`
`}`
		each neighbor of the current node if there is a cheaper edge to connect that neighbor to the tree being built and, if there is, updates that connection.

`while (vOrig != -1) {`
`visited[vOrig] = true;`
`Node v = g.vertex(vOrig);`
`List<Node> adj = new ArrayList<>(g.adjVertices(v));`
 	
`adj.sort(Comparator.comparingInt(g::key));`
`for (Node vAdj : adj) {`
`int adjKey = g.key(vAdj);`
`double distance = g.edge(v, vAdj).getDistance();`
`if (!visited[adjKey] && dist[adjKey] > distance) {`
`dist[adjKey] = distance;`
`path[adjKey] = vOrig;`
`}`
`}`
`vOrig = getVertMinDist(dist, visited);`
`}`
	$O(n^2 + e \log n)$	While loop that incrementally builds the minimum spanning tree by repeatedly adding the closest non-visited node and updating the minimum connection costs to its adjacent nodes.

Total temporal complexity:	**$O(n^2 + e \log n)$**	-

***getVertMinDist* method from *Algorithms* class**		
Code line	**Complexity**	**Description**

`for (int i = 0; i < dist.length; i++){`
`if (!visited[i] && dist[i] < min) {`
`min = dist[i];`
`v = i;`
`}`
`}`
	O(n)	For loop that selects the unvisited node with the lowest dist value and it will be the next one to enter the MST.

***getVertMinDist* method from *Algorithms* class**		
Total temporal complexity:	**O(*n*)**	-

***buildMst* method from *Algorithms* class**		
Code line	**Complexity**	**Description**
`Graph<Node, Double> mst = new MapGraph<>(false);`	O(1)	Declaration of a new unidirected graph.
`for (int i = 0; i < g.numVertices(); i++) { mst.addVertex(g.vertex(i)); }`	O(n)	For loop that adds all vertices into the new graph.
`for (int i = 0; i < path.length; i++){ if (path[i] != -1) { Node v = g.vertex(i); Node u = g.vertex(path[i]); mst.addEdge(u, v, 0.0, dist[i], 0); } }`	O(v)	For loop that for each node (except the root), the loop adds to the MST the edge that connects it to its predecessor, forming the final tree.
Total temporal complexity:	**O(*n + v*)**	-

Determinism Analysis:

Since:

- The iteration over the neighbours of each node is made over an ordered-by-node-index list, what guarantees the neighbours processing order is always the same,

- The next node is selected by *getVertMinDist*, which returns the non-visited node with the smallest distance, breaking ties using the node index and

- The method *buildMst* depends on pre-ordered arrays and runs through their fixed order,

it's concluded that the implemented Prim's Algorithm is deterministic, which means that when a graph comes as input, the MST generated from it will be the exact same in every execution.